MINI PROJECT REPORT

PHISHING WEBSITE DETECTION USING DEEP NEURAL NETWORKS

A Cybersecurity Approach for Identifying Malicious and Fraudulent Web Pages

Submitted by

Name: ARJUN GIRI

Register Number: 711524BCS011

Class & Section: III- CSE- B

Department of Computer Science and Engineering

ABSTRACT

Phishing is a cyber-attack in which criminals create fake websites that imitate trusted organizations, such as banks, online stores, and email providers, in order to steal passwords, card details, and other sensitive information. Blacklists and manual reporting can block known malicious sites, but new phishing pages appear constantly and may remain active for only a few hours. Phishing Website Detection Using Deep Neural Networks proposes a system that analyses the characteristics of a web address and web page and predicts whether the site is legitimate or phishing, without depending only on a list of previously reported sites.

The proposed approach uses a Deep Neural Network (DNN), a feed-forward network with several hidden layers, to learn patterns from features such as URL length, presence of an IP address, use of special characters, domain age, HTTPS usage, and page-level indicators. Records are collected from a labelled dataset of legitimate and phishing websites, cleaned, scaled, and divided into training, validation, and testing sets. The DNN is trained to classify each website, and techniques such as dropout, batch normalization, and early stopping are used to reduce overfitting. The trained model can be connected to a simple interface where a user enters a URL and receives a predicted class with a probability score.

The project is intended as a decision-support tool that adds a layer of protection rather than a replacement for secure browsing practices, blacklists, or security teams. Its reliability depends on dataset quality, how recent the training data is, and whether attackers change their techniques. Evaluation can be performed using accuracy, precision, recall, F1-score, and a confusion matrix. The system may support faster screening of suspicious links, while further testing on current real-world data is required before practical deployment.

KEYWORDS

Deep Learning, Deep Neural Network, Phishing Detection, Malicious URL, Cybersecurity, Binary Classification, Feature Extraction, Dropout Regularization.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Existing System
6. Proposed System
7. System Requirements
8. Dataset Description and Preparation
9. Methodology

10. System Architecture
11. Deep Neural Network Model Design
12. Implementation Plan
13. Evaluation Metrics
14. Expected Results
15. Advantages
16. Limitations
17. Applications
18. Future Enhancements
19. Conclusion
20. References

INTRODUCTION

Phishing attacks exploit human trust rather than technical weaknesses. An attacker sends an email, message, or advertisement containing a link to a fake website that looks like a genuine login or payment page. When the victim enters their details, the information is captured by the attacker. Such attacks can lead to financial loss, identity theft, and unauthorized access to accounts, and they affect both individuals and organizations.

In conventional practice, protection relies on blacklists of known malicious addresses, browser warnings, spam filters, and user awareness. Blacklists are useful but can only block sites that have already been reported, so newly created phishing sites may go undetected. Rule-based heuristics are easier to evade, because attackers can adjust their pages once the rules are known. Deep learning can automatically learn complex, non-linear combinations of features from examples rather than relying entirely on manually written rules.

A deep-neural-network-based phishing detection system accepts a website address, extracts a fixed set of features, applies the same preprocessing used in training, and predicts a class learned from labelled examples. The output is usually Legitimate or Phishing. The result is a prediction, not a guaranteed verdict; suspicious or high-impact cases should be confirmed using additional security tools or by a security professional.

PROBLEM STATEMENT

Phishing websites are created quickly, closely imitate genuine sites, and are difficult for users to recognise by sight alone. Blacklist-based protection is slow to react to new sites, and manual inspection cannot scale to the volume of links people receive. The problem addressed by this project is to develop a classification system that uses features of websites to predict whether a site is

legitimate or phishing accurately and efficiently. The system should provide a clear result, be straightforward to use, and be evaluated on websites that were not used during training.

OBJECTIVES

- To study the use of deep learning and machine learning for phishing website detection.
- To prepare a labelled dataset of legitimate and phishing websites with URL and page-based features.
- To preprocess data by handling missing values, encoding features, and scaling numerical features.
- To design and train a Deep Neural Network with suitable hidden layers, activation functions, and regularization.
- To classify a website as legitimate or phishing from its extracted features.
- To evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- To create a simple user interface for entering a URL and viewing the predicted result.
- To document limitations and encourage verification of important or uncertain predictions.

SCOPE OF THE PROJECT

The mini-project focuses on classifying websites into two categories, legitimate and phishing, using features available in the training dataset. It may begin with a ready-made feature dataset before being extended to extract features directly from live URLs. The system can be developed as a Python prototype and connected to a lightweight web interface or browser extension. It does not remove malicious content, track attackers, block network traffic, or replace antivirus software, firewalls, and security professionals.

The system's performance should be tested on websites that were not used for training and, where possible, on recently collected phishing and legitimate URLs. The scope should clearly state which features and types of phishing are supported, because a model cannot reliably detect attack styles that differ greatly from its training data.

EXISTING SYSTEM

Existing approaches include blacklists such as those used by web browsers, user awareness training, spam and email filters, and rule-based heuristic detection. Blacklists provide quick protection against known threats and remain useful. However, they depend on prior reporting and may miss new sites, while fixed rules and simple models can be evaded when attackers change their methods and may not capture complex interactions among many features.

Drawbacks of the Existing System

- Blacklists cannot detect newly created phishing websites that have not yet been reported.
- Phishing sites often live for a very short time, leaving little time for manual reporting.
- Rule-based heuristics may be bypassed once attackers learn the rules.

- Legitimate and phishing sites can share similar features, making classification challenging.
- A system trained on old data may not perform well on newer attack techniques.

PROPOSED SYSTEM

The proposed system uses a Deep Neural Network to learn patterns directly from website features. During training, the model receives websites with known labels and adjusts its weights to reduce classification error. During prediction, a new URL is converted into the same feature set, and the trained model returns a phishing probability. The class with the higher probability is displayed along with a confidence estimate.

A typical workflow includes URL input, validation, feature extraction, preprocessing, model inference, result display, and optional feedback collection. Regularization techniques such as dropout and early stopping help the model generalize to unseen sites. The final results must be measured on a separate test set.

Main Modules

- URL Input: accepts a website address from a dataset file or user entry.
- Feature Extraction: derives URL-based, domain-based, and page-based features from the address.
- Model Training: trains a DNN with hidden layers using labelled legitimate and phishing examples.
- Prediction: generates a probability for each class and selects the higher-scoring class.
- Result Display: presents the predicted class and, if appropriate, the model's confidence score.
- Evaluation: compares predictions with the true labels and calculates performance metrics.

SYSTEM REQUIREMENTS

Hardware Requirements

- Computer or laptop with a modern multi-core processor.
- At least 8 GB RAM recommended; more memory is helpful for larger datasets.
- Adequate storage for the dataset, source code, and trained model.
- GPU optional; the feature dataset is small, so a CPU is usually sufficient for training.

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Programming Language: Python 3.
- Libraries: TensorFlow/Keras or PyTorch, NumPy, Pandas, Matplotlib, scikit-learn, and a URL-parsing library such as tldextract.
- Development Environment: Visual Studio Code, Jupyter Notebook, or Google Colab.
- Optional Interface: Flask or Streamlit for a simple web application, or a browser extension.

DATASET DESCRIPTION AND PREPARATION

A labelled dataset is required to train and evaluate the model. Commonly used research datasets include the UCI Phishing Websites dataset, which provides numerical features for thousands of legitimate and phishing sites, and the Web Page Phishing Detection dataset, which also includes URL and content-based features. Dataset selection must be based on the project's target features, licence, and suitability for the intended use. A dataset should not be described as representing all current phishing techniques unless that has been verified.

Dataset Organization

A convenient structure is a single table in which each row represents one website and each column represents a feature, with one column serving as the label. For example, features may include URL length, presence of an IP address or the "@" symbol, number of subdomains, HTTPS usage, and domain age. The column names should match the actual dataset and must not be invented after training.

Data Splitting

Divide the dataset into training, validation, and testing subsets. A possible split is 70% training, 15% validation, and 15% testing, although other ratios may be appropriate. The training set updates model parameters, the validation set helps tune hyperparameters and monitor overfitting, and the test set is reserved for final evaluation. Duplicate URLs should not appear in different subsets, and scaling values must be computed on the training set only, because otherwise measured performance can be inflated.

Data Preprocessing

- Handle missing values and remove duplicate or malformed URLs and inconsistent records.
- Encode categorical or ternary-valued features (for example -1, 0, 1) in a form suitable for the network.
- Scale numerical features using standardization or min-max normalization.
- Check for incorrect labels, outliers, and class imbalance between phishing and legitimate sites.
- Apply class weights or resampling only to training data when classes are imbalanced, and keep the test set untouched.

METHODOLOGY

Data Collection

Collect website records from a public dataset or from reliable sources of phishing and legitimate URLs. Record the dataset source, version, feature list, and any usage conditions. Remove duplicates, and include enough examples of both classes to support learning rather than relying on a very small number of sites. Phishing pages should be handled safely and never opened directly on an unprotected system.

Preprocessing and Feature Extraction

URL-based features such as length, number of dots, special characters, and use of IP addresses are extracted, along with domain-based features such as domain age and HTTPS use. Missing values are handled and numerical attributes are scaled so that features with large ranges do not dominate learning. Feature selection or correlation analysis can remove redundant attributes, and every feature must be obtainable at prediction time.

Model Selection

A Deep Neural Network is suitable because stacked hidden layers can learn non-linear combinations of features that simple rules may miss. For a tabular feature dataset of this kind, a fully connected network with two to four hidden layers is a reasonable starting point. Its results should be compared with baseline models such as logistic regression or random forest, so that the benefit of the deep model is measured rather than assumed.

Training

During training, the model predicts a probability for each website. A loss function, such as binary cross-entropy, measures the difference between predicted scores and true labels. An optimizer such as Adam updates model weights through backpropagation. Training should be monitored using validation loss and validation accuracy. Dropout, batch normalization, early stopping, and model checkpointing help reduce overfitting and retain the best validation model.

Prediction

For a new URL, the application extracts the same features and performs the same scaling used during training. The model generates a phishing probability, and a decision threshold, commonly 0.5, converts it into a class. The threshold can be adjusted depending on whether missing a phishing site or wrongly flagging a genuine site is more costly. A high confidence score does not necessarily mean the prediction is correct, especially for unfamiliar attack styles.

Output

The application displays the entered URL, the predicted class, and an optional confidence score. A responsible implementation should also show a message that the output is an automated prediction and that users should avoid entering personal information on uncertain sites and should verify important cases through trusted sources.

SYSTEM ARCHITECTURE

The high-level architecture consists of the following stages:

1. User or Dataset → Website URL
2. Feature Extraction and Preprocessing → Scaled Feature Vector
3. Trained Deep Neural Network → Phishing Probability
4. Post-processing → Legitimate or Phishing Class
5. User Interface → Prediction, Confidence Estimate, and Advisory Warning

The training pipeline is separate from the prediction pipeline. Training uses labelled websites and validation feedback to create a saved model file and a saved scaler. The prediction application loads both and uses them to classify new URLs. This separation makes it easier to update the model as new phishing data becomes available without redesigning the interface.

DEEP NEURAL NETWORK MODEL DESIGN

A basic DNN may contain the following conceptual layers. The exact number of neurons and layers should be selected through experiments rather than treated as a universal configuration.

- Input Layer: accepts the preprocessed feature vector, with one unit per feature.
- Dense Hidden Layer 1: a fully connected layer that learns combinations of input features.
- Activation Layer: ReLU introduces non-linearity so the model can learn complex patterns.
- Batch Normalization: stabilizes and speeds up training by normalizing layer outputs.
- Dropout Layer: randomly disables neurons during training to reduce overfitting.
- Additional Hidden Layers: progressively smaller dense layers learn higher-level patterns.
- Output Layer: uses a single sigmoid unit to give the probability that a site is phishing.
- Compilation: the model is compiled with binary cross-entropy loss, the Adam optimizer, and evaluation metrics.

For binary classification, the output layer contains a single unit with sigmoid activation. If more than two categories are used, such as legitimate, phishing, and suspicious, the output layer contains one unit per class with softmax activation. Labels and outputs must use compatible formats, and the same preprocessing must be applied at training and prediction time.

IMPLEMENTATION PLAN

The project can be implemented in phases so that each part can be tested independently.

1. Phase 1 — Define the problem, features, dataset, and evaluation plan.
2. Phase 2 — Download or organize the labelled dataset and inspect it for missing and duplicate records.
3. Phase 3 — Split the data and build the preprocessing pipeline.
4. Phase 4 — Train a baseline model and a Deep Neural Network.
5. Phase 5 — Evaluate the models on the validation and test sets and inspect errors.
6. Phase 6 — Save the trained model and create a URL-input interface.
7. Phase 7 — Test the full workflow with unseen URLs and document limitations.

Example Pseudocode

```
Load labelled phishing website dataset
Clean data and remove duplicate records
Split data into training, validation, and testing sets
Fit scaler on training data and transform all sets
Build DNN with dense, dropout, and sigmoid output layers
Train model and monitor validation metrics
Save the best model and the scaler
```

```
Evaluate saved model on the held-out test set
For each new URL:
    Validate the URL format
    Extract and scale the features
    Predict phishing probability using saved model
    Display predicted class and suitable warning
```

EVALUATION METRICS

The model should be evaluated on a held-out test set. Accuracy alone can be misleading when classes are imbalanced, since a model that predicts only the majority class may still appear accurate, so class-wise metrics and a confusion matrix should also be reported. In phishing detection, the cost of errors differs: missing a phishing site is usually more harmful than wrongly flagging a genuine one.

Accuracy

Accuracy = (Number of correct predictions / Total number of predictions) × 100.

Precision

Precision = TP / (TP + FP). Precision indicates how many sites predicted as phishing were actually phishing. Here, TP is true positives and FP is false positives. Low precision means many genuine sites are wrongly flagged.

Recall

Recall = TP / (TP + FN). Recall indicates how many actual phishing sites were correctly identified. FN means false negatives. High recall for the phishing class is important so that dangerous sites are not missed.

F1-Score

F1-score = 2 × (Precision × Recall) / (Precision + Recall). It balances precision and recall. Macro-averaged metrics give each class equal weight, while weighted averages account for class frequencies.

Confusion Matrix

A confusion matrix compares actual classes with predicted classes. It helps identify false positives, where genuine sites are flagged, and false negatives, where phishing sites are missed. For example, well-designed phishing pages that closely copy genuine sites may be misclassified more frequently.

Reporting Results

Do not insert assumed accuracy values into the final report. Train the model and record the actual test results. A results table may be completed as follows:

Metric	Measured result
Test accuracy	_____
Macro precision	_____

Macro recall	_____
Macro F1-score	_____
Number of test samples	_____

EXPECTED RESULTS

The expected outcome is a working prototype that can distinguish legitimate websites from phishing websites based on the features represented in its training data. The model should learn patterns associated with each class and provide a predicted label for a previously unseen URL. Actual performance will depend on dataset size, label quality, class balance, chosen features, model configuration, and the similarity of test data to training data. Results must be measured experimentally and reported honestly.

The project should also produce an evaluation summary, such as accuracy and F1-score, a confusion matrix, a comparison with baseline models, and examples of correct and incorrect predictions. Error analysis is important because it can reveal weaknesses caused by outdated data, unusual URL formats, missing features, or new phishing techniques.

ADVANTAGES

- Can provide quick preliminary screening of suspicious websites.
- Learns non-linear relationships among features automatically from training data.
- Can detect new phishing sites that are not yet on any blacklist, if they resemble learned patterns.
- Can be connected to a simple web interface or browser extension.
- Can support user awareness and reduce the risk of credential theft.
- Regularization techniques help the model generalize to unseen websites.

LIMITATIONS

- Performance depends on dataset size, diversity, recency, and label accuracy.
- Attackers continuously change techniques, so the model may become outdated and needs periodic retraining.
- Some features, such as domain age, may need external lookups that can be slow or unavailable.
- Small feature-based datasets may not benefit much from deep models over simpler baselines, so comparison is necessary.
- A model confidence score is not a guarantee of correctness.
- False positives may block genuine sites, and false negatives may expose users to harm.
- Attackers may deliberately craft pages to evade the model.

These limitations can be reduced—but not completely removed—through recent and diverse data, careful feature selection, threshold tuning, regular retraining, combination with blacklists and other security tools, and periodic monitoring after deployment.

APPLICATIONS

- Browser extensions that warn users about suspicious websites.
- Email and messaging security tools that scan links before delivery.
- Research prototypes for cybersecurity decision-support systems.
- Educational demonstrations of deep learning and cybersecurity.
- Support for security teams when reviewing reported suspicious links.

FUTURE ENHANCEMENTS

- Expand the dataset to cover more phishing techniques, languages, and recently reported sites.
- Develop a browser extension that checks pages in real time.
- Use character-level or sequence models such as CNN or LSTM directly on URL strings.
- Add page-content, HTML structure, and visual similarity features such as logo and layout comparison.
- Retrain the model periodically on fresh data and monitor performance over time.
- Add explainability methods such as SHAP or LIME to show which features influenced each prediction.
- Combine the model with blacklists and WHOIS or certificate checks for stronger protection.
- Add adversarial testing to measure resistance against evasion attempts.

CONCLUSION

Phishing Website Detection Using Deep Neural Networks demonstrates how deep learning can be applied to a cybersecurity problem. By training a DNN on labelled legitimate and phishing websites, a system can learn patterns in URL and page features and classify new sites as safe or suspicious. The project combines dataset preparation, feature extraction, model training, evaluation, and a user-facing prediction interface.

The system can offer a useful additional layer of protection, but its predictions should not be treated as definitive. Reliable deployment requires testing on current real-world data, reporting measured performance, tuning the decision threshold, regular retraining, and using it together with other security measures. With these safeguards, the project provides a practical mini-project for learning deep learning while addressing a meaningful problem in online safety.

REFERENCES

1. Mohammad, R. M., Thabtah, F., and McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. Neural Computing and Applications, 25(2), 443–458. https://archive.ics.uci.edu/dataset/327/phishing+websites
2. Hannousse, A., and Yahiouche, S. (2021). Towards benchmark datasets for machine learning based website phishing detection: An experimental study. Engineering Applications of Artificial Intelligence, 104, 104347.

3. Goodfellow, I., Bengio, Y., and Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/
4. Kingma, D. P., and Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980. https://arxiv.org/abs/1412.6980
5. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15, 1929–1958.
6. TensorFlow. Keras guide: training and evaluation, and basic classification tutorials. https://www.tensorflow.org/tutorials/keras/classification